Lecture 5: Recurrent Neural Network (RNN)

Yaxin Bi
School of Computing
University of Ulster

1

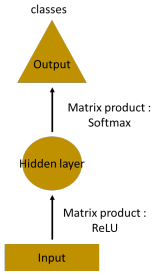
Contents

2

Use an MLP to Predict Next Word

Use a multilayer perceptron (MLP) to predict the next word in a sentence. In the simplest form, we have an input layer, a hidden layer and an output layer

- the input layer receives the input
- the hidden layer activations are applied
- and then finally receive the output.



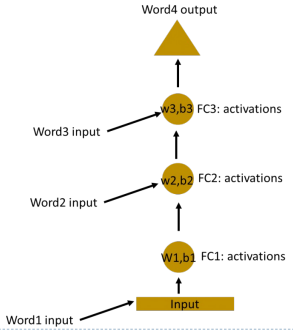
3

Use an MLP to Predict Next Word(cont'd)

- Let's have a deeper network with multiple hidden layers
  - the input layer receives the input,
  - the first hidden layer activations are applied
  - and then these activations are sent to the next hidden layer
  - and successive activations through the layers to produce the output.
- Each hidden layer is characterized by its own weights and biases.
- Each hidden layer has its own weights and activations, they behave independently.
- The objective is to identify the relationship between successive inputs.

4

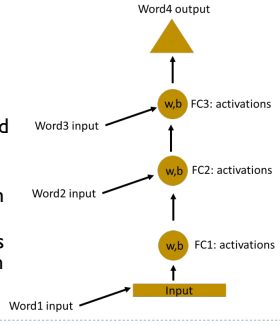
Use an MLP to Predict Next Word(cont'd)



5

Combine Hidden Layers

- In the current structure, the weights and bias of these hidden layers are different.
- Each of these layers behave independently and cannot be combined together.
- To combine these hidden layers together, we have to have the same weights and bias for these hidden layers.



6

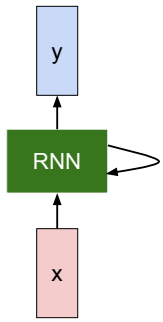
Key Idea of Recurrent Neural Networks (RNN)

- We now combines these layers together
  - the weights and bias of all the hidden layers is the same
  - All these hidden layers are rolled in together in a single recurrent layer.
- This is like supplying the input to the hidden layer.
- At all the time steps, weights of the recurrent neuron would be the same since its a single neuron now.
- So a recurrent neuron stores the state of a previous input and combines with the current input thereby preserving some relationship of the current input with the previous input.

7

7

Illustration of Key Idea of RNNs



RNNs have an "internal state" that is updated as a sequence is processed

8

8

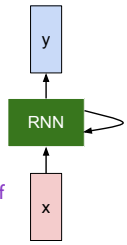
Formula for the Current State

- We can process a sequence of vectors x by applying a recurrence formula at every time step:

$$h_t = f_W(h_{t-1}, x_t)$$

new state — old state — input vector at some time step

some function with parameters W



- Notice: the same function and the same set of weights and bias are used at every time step.

9

9

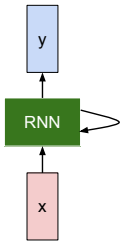
Activation Function for a Simple RNN (Vanilla RNN)

- The state consists of a single "hidden" vector h_t:

$$h_t = f_W(h_{t-1}, x_t)$$

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$



where the activation function is tanh, the weight at the recurrent neuron is W_{hh} and the weight at the input neuron is W_{xh}, the output state is y_t.

10

10

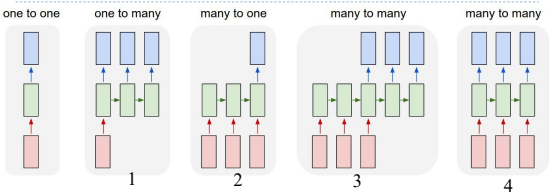
Summary of Steps in a Recurrent Neuron

- A single time step of the input x_t is supplied to the network
- Then calculate its current state using a combination of the current input and the previous state i.e. we calculate h_t
- The current h_t becomes h_{t-1} for the next time step
- We can go as many time steps as the problem demands and combine the information from all the previous states
- Once all the time steps are completed the final current state is used to calculate the output y_t
- The output is then compared to the actual output and the error is generated
- The error is then backpropagated to the network to update the weights and the network is trained

11

11

Types of Computational Graphs

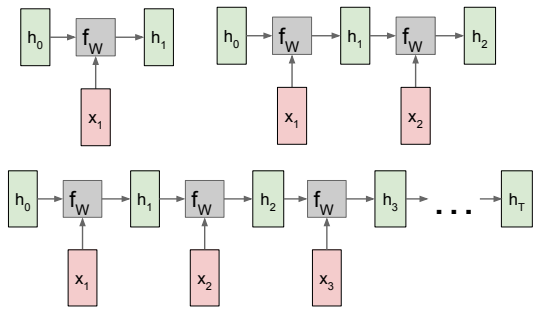


1. Image Captioning: image → sequence of words
2. Sentiment Classification: sequence of words → sentiment
3. Machine Translation: sequence of words → sequence of words
4. Video: classification on frame level

12

12

Many to One Structure

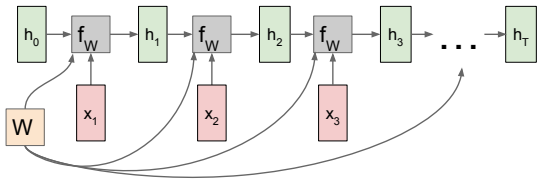


13

13

Many to One Structure (cont'd)

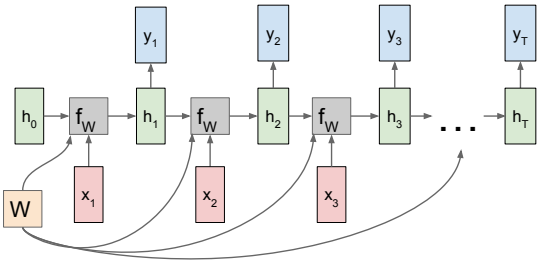
- Re-use the same weight matrix at every time-step



14

14

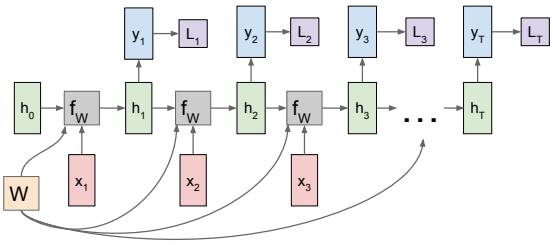
Many to Many Structure



15

15

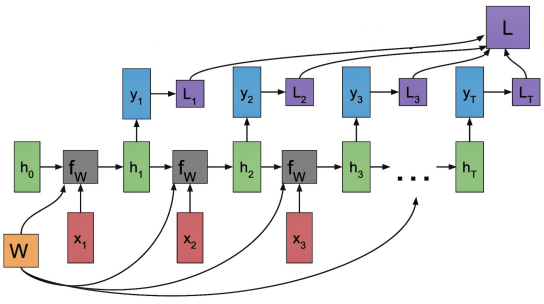
Many to Many Structure with Loss



16

16

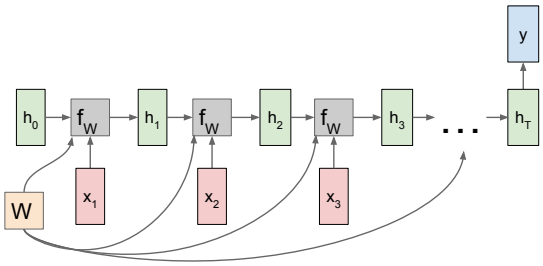
Many to Many Structure with a Combined Loss



17

17

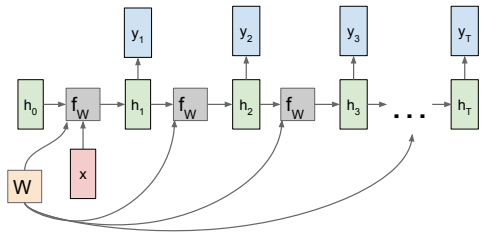
Many to One Structure



18

18

One to Many Structure

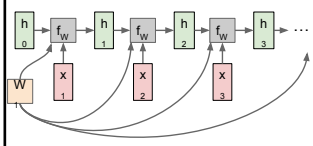


19

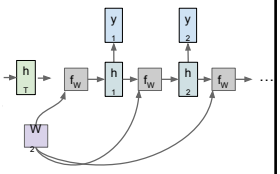
19

Sequence to Sequence: Many-to-One + One-to-Many

Many to one: Encode input sequence in a single vector



One to many: Produce output sequence from single input vector

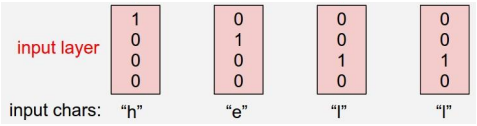


20

20

Example: Character-level Language Model

Character-level Language Model
Vocabulary: [h,e,l,o]
Training sequence: "hello"



The inputs are one hot encoded, the entire vocabulary is {h, e, l, o}, hence it is easy to generate one hot encoded the inputs.

21

21

Example: initialize the weight Wxh

- Now the input neuron would transform the input to the hidden state using the weight wxh. We randomly initialize the weights as a 3*4 matrix below

wxh			
0.287027	0.84606	0.572392	0.486813
0.902874	0.871522	0.691079	0.18998
0.537524	0.09224	0.558159	0.491528

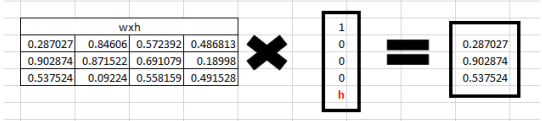
22

22

Example: compute Wxh*Xt

Step 1:

Now for the letter "h", for the the hidden state we would need Wxh*Xt. By matrix multiplication, we get it as –



23

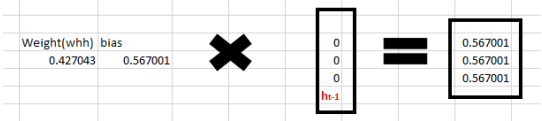
23

Example: compute Whh*ht-1

Step 2:
Now moving to the recurrent neuron, we have Whh as the weight which is a 1*1 matrix as 0.427043 and the bias which is also a 1*1 matrix as 0.567001

For the letter "h", the previous state is [0,0,0] since there is no letter prior to it.

So to calculate -> (whh*h_{t-1}+bias)



24

24

Example: compute Whh*ht-1

Step 2:
Now moving to the recurrent neuron, we have Whh as the weight which is a 1*1 matrix as 0.427043 and the bias which is also a 1*1 matrix as 0.567001

For the letter "h", the previous state is [0,0,0] since there is no letter prior to it.

So to calculate -> (whh*h_{t-1}+bias)

24

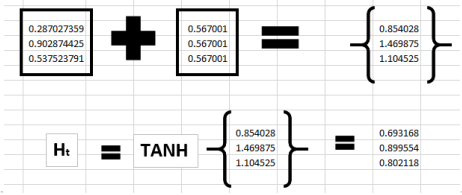
24

Example: compute Whh*ht-1 + Wxh*Xt

Step 3:
Now we can get the current state as –

$$h_t = \tanh (W_{hh}h_{t-1} + W_{xh}x_t)$$

Since for h, there is no previous hidden state we apply the tanh function to this output and get the current state –



25

25

Example: compute tanh

Step 4:
Now we go on to the next state. "e" is now supplied to the network. The processed output of h_t, now becomes h_{t-1}, while the one hot encoded e, is x_t. Let's now calculate the current state h_t.

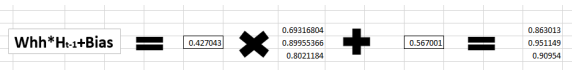
$$h_t = \tanh (W_{hh}h_{t-1} + W_{xh}x_t)$$

26

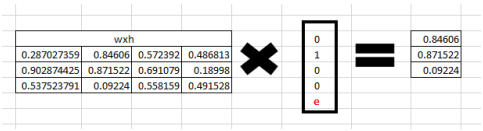
26

Example: compute tanh (cont'd)

Step 4:
Whh*h_{t-1} +bias will be –



Wxh*x_t will be –

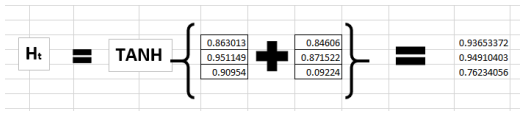


27

27

Example: compute tanh (cont'd)

Step 5:
Now calculating h_t for the letter "e",



Now this would become h_{t-1} for the next state and the recurrent neuron would use this along with the new character to predict the next one.

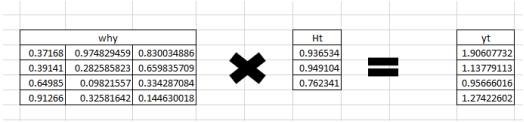
28

28

Example: compute the output

Step 6:
At each state, the recurrent neural network would produce the output as well. Let's calculate y_t for the letter e.

$$y_t = W_{hy}h_t$$

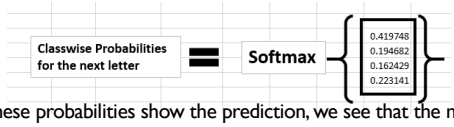


29

29

Example: compute softwax

Step 7:
The probability for a particular letter from the vocabulary can be calculated by applying the softmax function. Thus we shall have softmax(y_t)



These probabilities show the prediction, we see that the model says that the letter after "h" should be "e", since the highest probability is for the letter "h". Does this mean we have done something wrong? No, so here we have hardly trained the network. The example just shows its two letters.

30

30

Backpropagation in a Recurrent Neural Network

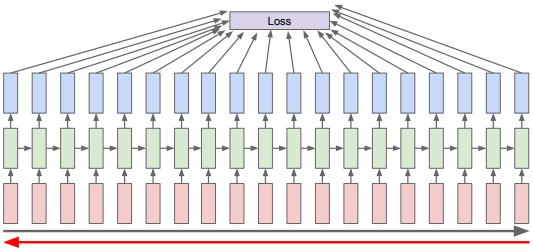
- To imagine how weights would be updated in case of a recurrent neural network, might be a bit of a challenge. So to understand and visualize the back propagation, let's unroll the network at all the time steps.
- In an RNN we may or may not have outputs at each time step.
- In case of a forward propagation, the inputs enter and move forward at each time step.
- In case of a backward propagation, we are going back in time to change the weights, hence we call it the Backpropagation through time

31

31

Backpropagation Through Time

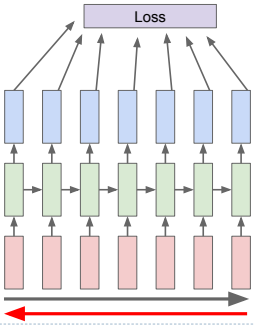
- Forward through entire sequence to compute loss, then backward through entire sequence to compute gradient



32

32

Truncated Backpropagation through time

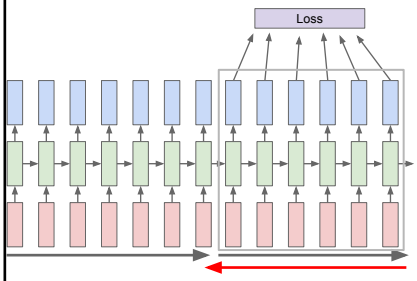


- Run forward and backward through chunks of the sequence instead of whole sequence

33

33

Truncated Backpropagation through time (cont'd)

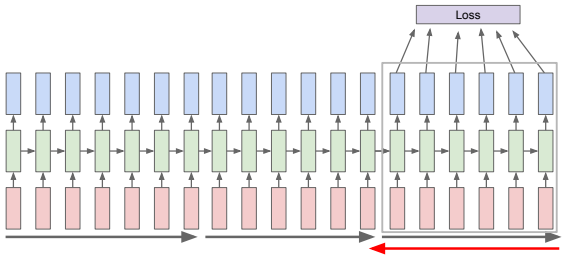


- Carry hidden states forward in time forever, but only backpropagate for some smaller number of steps

34

34

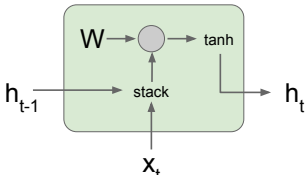
Truncated Backpropagation through time (cont'd)



35

35

Vanilla RNN Gradient Flow



Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994

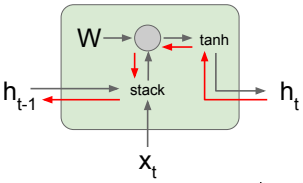
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$
$$= \tanh\left((W_{hh} \quad W_{hx})\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$
$$= \tanh\left(W\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$

36

36

Vanilla RNN Gradient Flow (cont'd)



Backpropagation from h_t to h_{t-1} multiplies by W (actually W_{hh}^T)

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$
$$= \tanh\left((W_{hh} \quad W_{hx})\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$
$$= \tanh\left(W\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$

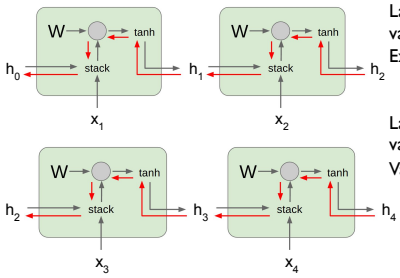
Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

37

37

Vanilla RNN Gradient Flow (cont'd)

- Computing gradient of h_0 involves many factors of W (and repeated tanh)



Largest singular value > 1: Exploding gradients

Largest singular value < 1: Vanishing gradients

38

38